Bias-Variance Tradeoff and the Train-Test Split

Tim Book

GENERAL ASSEMBLY

Agenda

- The Bias-Variance Tradeoff
- What is model “complexity”?
- Detecting overfit via holdout techniques
- Holdout Technique #1: **Train-Test Split**
- Holdout Technique #2: ***k*-Fold Cross Validation**

© 2018 General Assembly

What is “Error”?

In modeling, we use the word **error** to refer to deviation between a prediction and its (often unknown) underlying true value.

 © 2018 General Assembly

Error is Error, Right?



Don’t confuse error with “residual”. There are actually three (3!) types of error. You can think of a residual as the sum of all 3 types.

 © 2018 General Assembly

The Three Types of Error

- **Bias** - This is the error attributable to our model's inability to adequately predict a value. You can think of this plainly as "model badness".

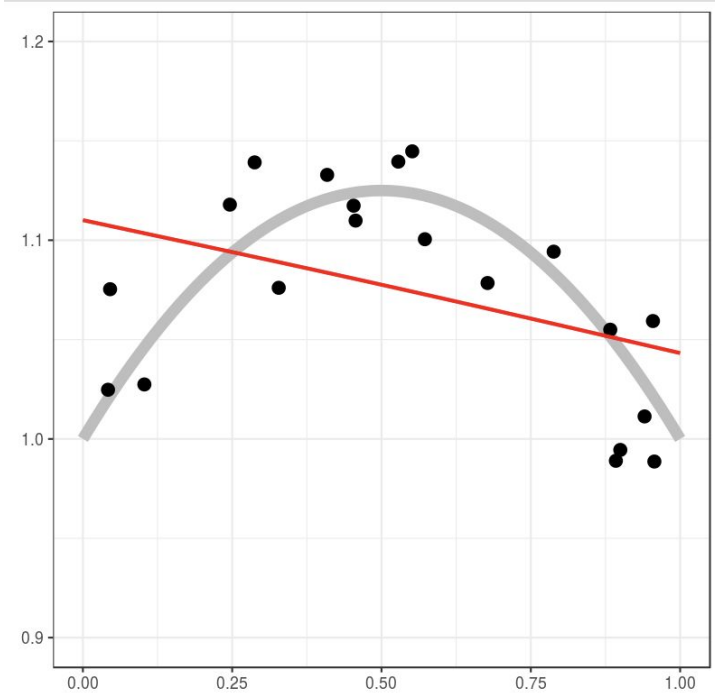
- **Variance** - This is the error attributable to the fact that you might just get a different *y* even with the same *x*s.

- **Irreducible error** - This is error attributable to the random nature of the universe. This can never be reduced or eliminated by statistical techniques.

 © 2018 General Assembly

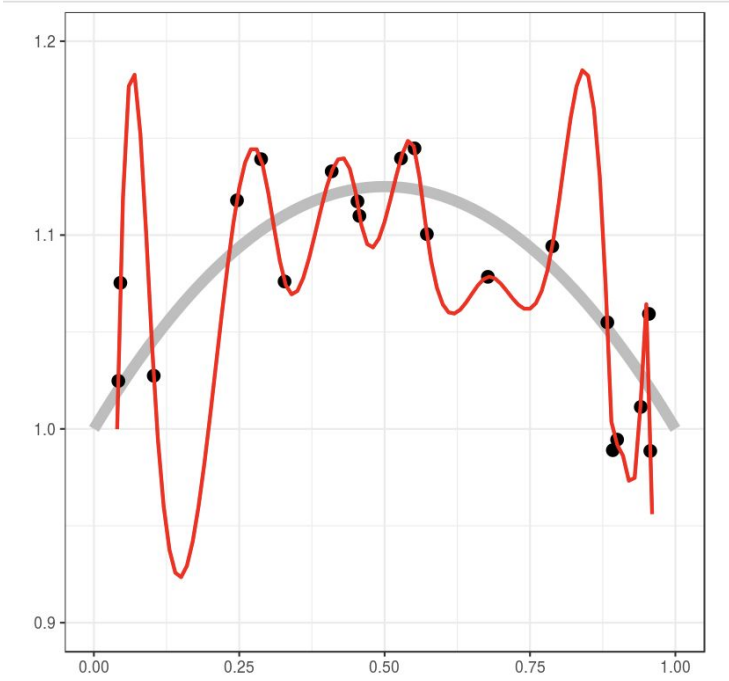
Bias

Our model (the red line) does not adequately capture the complexity of nature. The model does a poor job of modeling nature. There are forces at play we're not accounting for. This model is said to have **high bias** and is thus **underfit**.



Variance

In this case, the model (red line) is far too complicated for nature. It's not going to generalize to new data properly. This model is said to have **high variance** and is thus **overfit**.



 © 2018 General Assembly

Side-by-Side

Underfit Models

- Poor fit to training data
- Performs on new data just as well as old data (poorly)
- **Model is too simple** to model real life in this case.

Overfit Models

- Very good fit to training data
- Does not generalize to new data
- **Model is too complex.** Real life turned out to be simpler in this case.

 © 2018 General Assembly

What is model “complexity”?

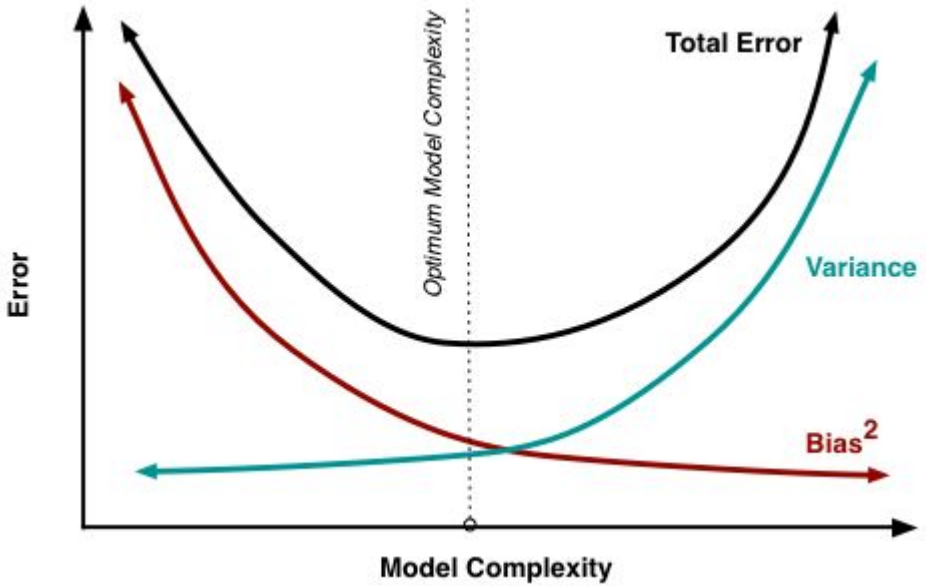
It’s really what it sounds like - how complicated is it to describe our model? Generally, more complicated models have a finer control on the model’s prediction, which can actually be a bad thing. Here are a few things that control model complexity:

- Number of variables in the model
- “Wiggliness” of variables included (polynomial fits: x, x^2, x^3, etc.)
- Level of regularization (a topic for future lessons)
- Model architecture (e.g., neural nets are more complicated than OLS)

 © 2018 General Assembly

The Whole Point

Our goal is to balance this **bias-variance tradeoff**! But how?!



http://scott.fortmann-roe.com/docs/docs/BiasVariance/biasvariance.png

Measuring the Bias-Variance Tradeoff

Measuring the BVT

It turns out, bias and variance are theoretical quantities and cannot be calculated directly from the data. We can **estimate** them, with **holdout techniques**, however!

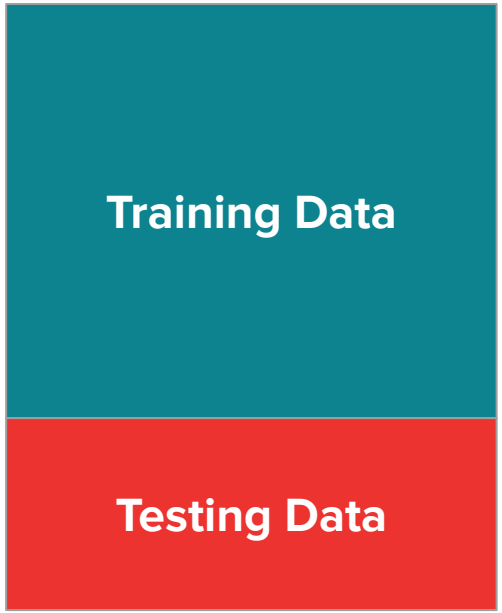
We'll start with the most famous holdout technique, the **train-test split**.

The Train-Test Split

Which Metrics to Choose?

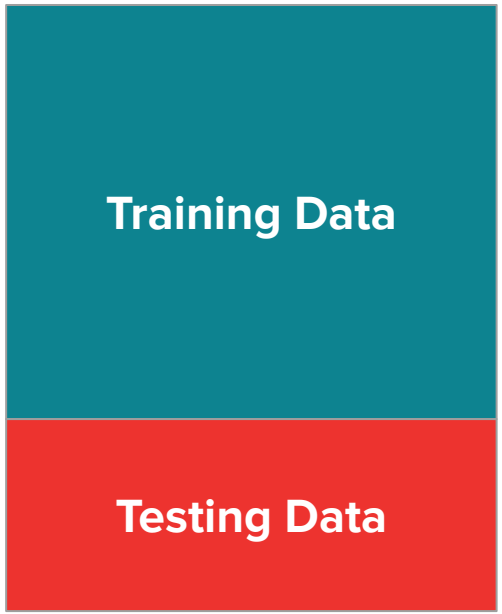
To estimate the bias-variance tradeoff, we'll need a metric to use as a benchmark. For regression problems, it's traditional to use the **MSE**.

The Train-Test Split



For this technique, we will divide our data into two pieces: the **training set** and the **testing set**.

The Train-Test Split

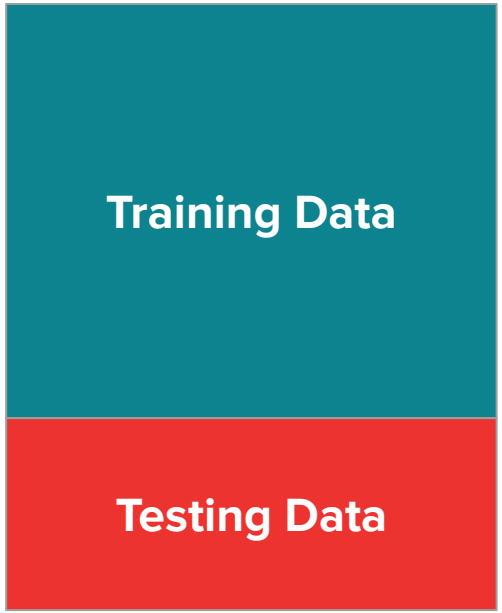


Training MSE:
150

A model is fit using only the training data and a **training metric** is computed.

 © 2018 General Assembly

The Train-Test Split



Training MSE:
150

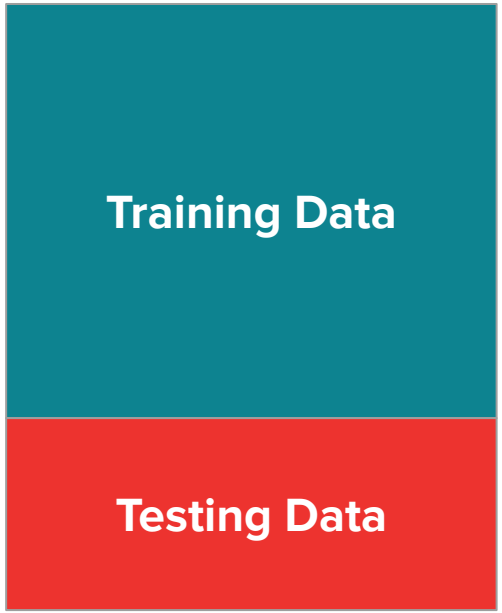
Testing MSE:
???

A model is fit using only the training data and a **training metric** is computed. Often it's the MSE.

Then, using the same model, we compute a **testing metric** to see how the model performs on new data.

 © 2018 General Assembly

The Train-Test Split



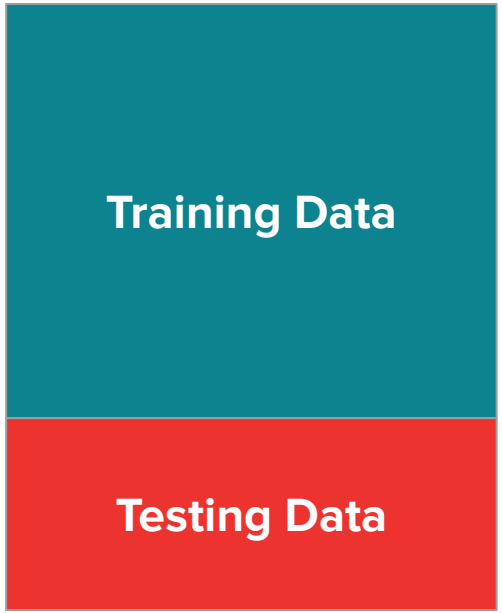
Training MSE:
150

Testing MSE:
250

If the testing metric is much higher (worse) than the training metric, that indicates our model is **overfit**, since it has not generalized to new data very well.

 © 2018 General Assembly

The Train-Test Split



Training MSE:
150

Testing MSE:
160

If the testing metric is “close enough” to the training metric, then our model is likely not overfit.

What is the best train-test ratio?

How much data should be held out for our testing data? There's actually no hard rule for this, but there's some good advice out there.

Generally, you want two things to be true:

- You want as much data as possible for model fitting
- You want enough testing data to adequately reflect all the types of data you might encounter in the real world

 © 2018 General Assembly

Train-Test Split Rule-of-Thumb

If you have **small data**, you'll want a higher proportion of data in the test set. For example, if you only have 200 samples, you *might* opt for a 50/50 split! This is because you need a representative test set.

If you have **large data**, you'll want a much larger proportion of data in the training data. For example, if you have 100M rows, you might opt for a 99/1 split! This is because you want as many training samples as possible, and 1M is more than sufficient for a representative test set.

In general, however, the default 75/25 or 70/30 split is usually fine.

 © 2018 General Assembly

Describe this model to me...

$$y \sim x_1 + x_2 + x_1 x_2 + x_3 + x_3^2 + x_4 + x_5$$

Train MSE = 12.3

Test MSE = 658.2



Describe this model to me...

$y \sim x_1 + x_2 + x_1x_2 + x_3 + x_3^2 + x_4 + x_5$

Train MSE = 12.3

Test MSE = 658.2

$y \sim x_1 + x_2$

Train MSE = 233.9

Test MSE = 252.1



How about this one on the left?

$y \sim x_3 + x_4$

Train MSE = 132.7

Test MSE = 200.5



$y \sim x_1 + x_2$

Train MSE = 233.9

Test MSE = 252.1

To the notebook!



k-Fold Cross-Validation

k-Fold Cross-Validation

There's another holdout technique that might be preferable in some situations: the ***k*-fold cross-validation** technique.

 © 2018 General Assembly

k-Fold Cross Validation

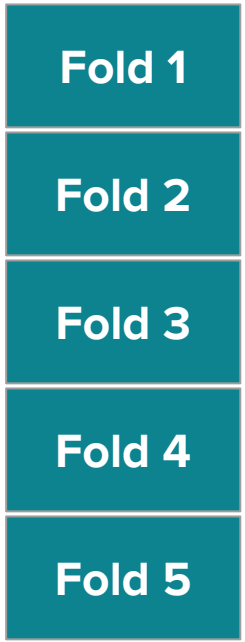


To begin, select a number *k*, and split your data into *k* **folds**.

This is not a tuning parameter, but *k* should be large enough so that averaging *k* values is meaningful, but also small enough so that splitting your data into *k* chunks doesn't make the folds too small.

 © 2018 General Assembly

k-Fold Cross Validation



We'll use $k = 5$ folds. Common choices are usually 5 or 10.

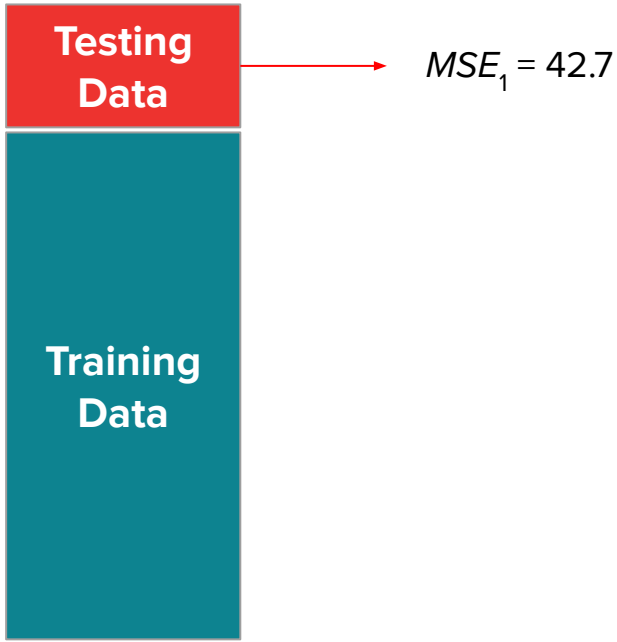
 © 2018 General Assembly

k-Fold Cross Validation



Next, we mark the first fold as “testing data” and the rest as “training”.

k-Fold Cross Validation

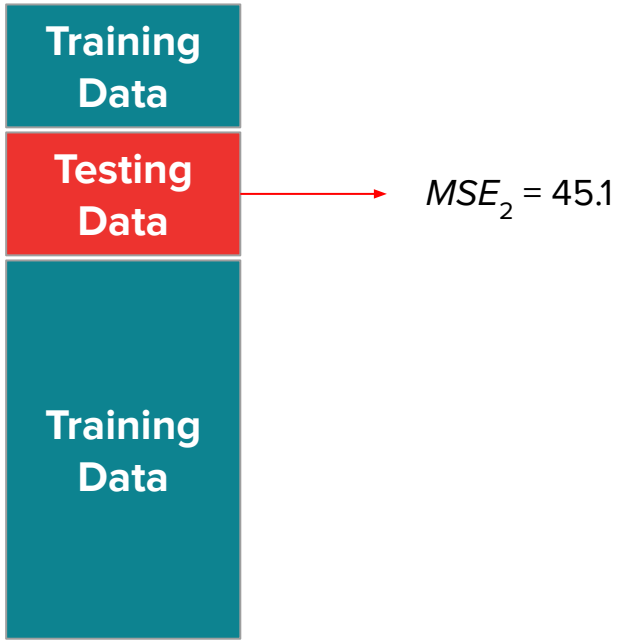


Next, we mark the first fold as “testing data” and the rest as “training”.

We fit a model to the training, and compute some error metric on the testing data. We’ll store this for later.

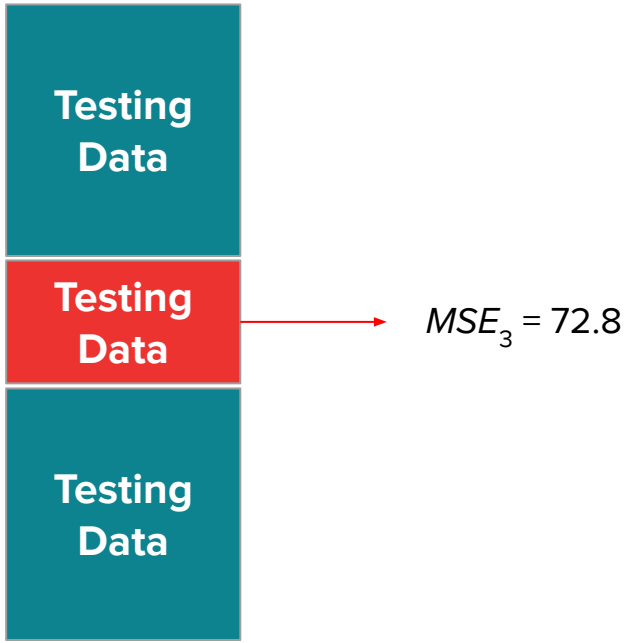
 © 2018 General Assembly

k-Fold Cross Validation



We repeat this process for the next fold. Fold 2 becomes the testing data, and folds 1, 3, 4, and 5 become training data.

k-Fold Cross Validation

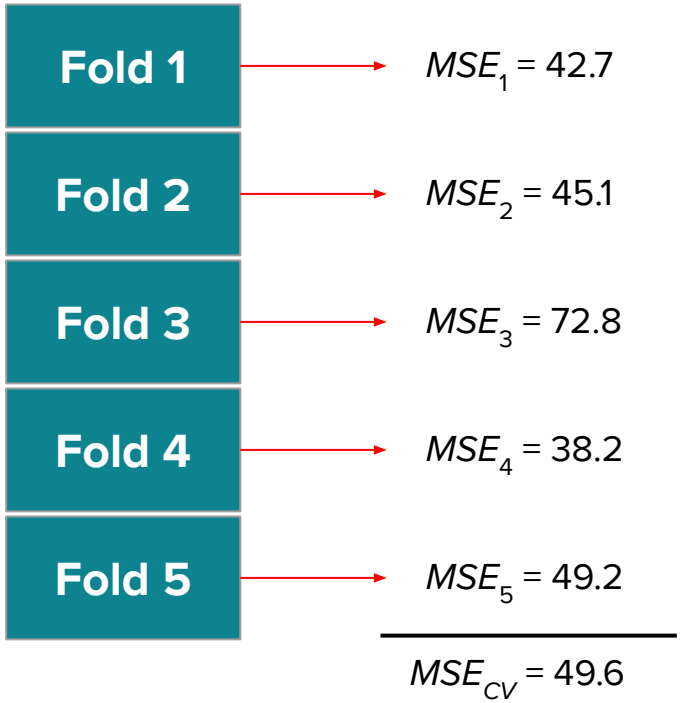


...and so on.

Sometimes, just due to chance, you'll get an outlying MSE.

 © 2018 General Assembly

k-Fold Cross Validation

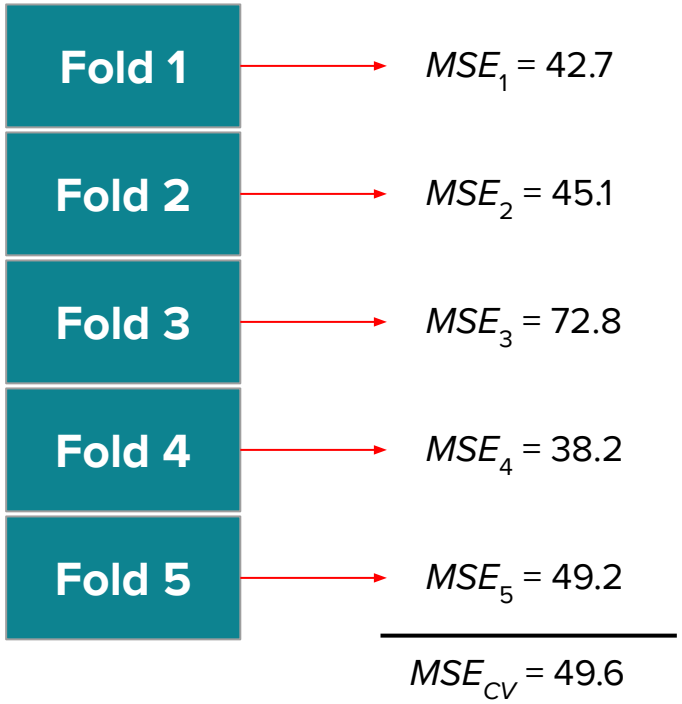


Eventually, we get one error metric per fold.

Finally, we average them together to get a **cross validated MSE**.

 © 2018 General Assembly

k-Fold Cross Validation



This value is our final result. This gives us a more robust measurement of model validity. We can compare this to a full-data MSE or a cross validated MSE from a model with different parameters!

 © 2018 General Assembly

To the notebook!



Why not always use this?

Advantages of CV:

- You get a measure of bias/variance that's more robust to outliers
- You don't "throw away" data - every observation gets to be used as part of model training

 © 2018 General Assembly

Why not always use this?

Disadvantages of CV:

- *Since the same data are used to compute the MSE in each fold, by averaging them you're actually introducing some bias into your estimate. (People typically ignore this though).*
- You need to carry out model fitting *k* times! This is a HUGE disadvantage!!

Sidenote: LOOCV

A special case of cross-validation is **Leave One Out Cross-Validation (LOOCV)**, which is when you let $k = n$. It's rarely used nowadays, but you might hear about it.

Ok, then what?

Ok, let's say you used TTS or CV to pick out a model architecture that meets your standards of quality and has a good bias-variance tradeoff. You're now tasked with deploying this model so that it can be used to make future predictions. What's your next step?

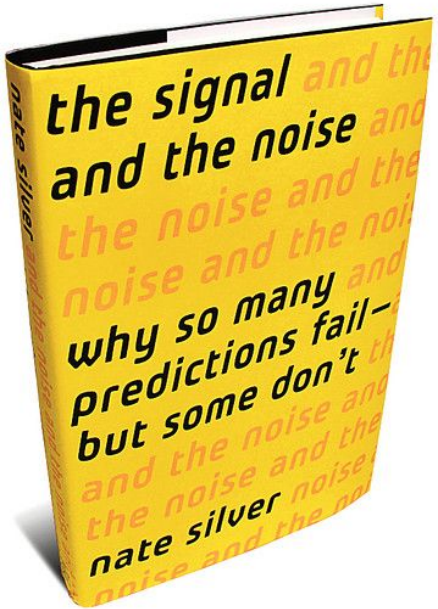


Discuss: What data do you use to fit your final model?

The Signal and the Noise

You can think about ML as modeling the **signal** between X and y and separating it from the **noise**.

If we overfit our model, we're just modeling noise! This happens a lot with inexperienced scientists of any field: including geologists!



 © 2018 General Assembly

Is Overfitting So Wrong?

Go back to that previous example. What do you think about overfitting?



Conclusions

- It is *always* mathematically possible to get a perfect model on your training data (think about this! How could you do it?).
- Having a good training MSE is unimpressive. We need to know our model can generalize to new data.
- We can *holdout* data from our model using either a train-test split or *k*-fold cross validation.
- Top priority in model prediction is to get a good testing MSE, as this measures how your model is *really* doing!

 © 2018 General Assembly